

Atlas Wspomnień

Architektury i technologie systemów internetowych

Aga Patro

1. Cel projektu.....	2
2. Zakres funkcjonalności.....	2
2.1. Charakterystyka użytkowników aplikacji.....	2
2.2. Szczegółowy opis funkcji.....	2
2.3. Wymagania niefunkcjonalne.....	3
3. Realizacja projektu.....	3
3.1. Architektura systemu.....	3
3.2. Wykorzystane technologie.....	4
3.2.1. Back-End.....	4
3.2.2. Front-End.....	4
3.2.3. Baza Danych.....	5
3.3. Baza danych.....	5
3.4. Server aplikacyjny.....	6
3.5. Aplikacja webowa.....	6
3.6. Deploy.....	7
4. Wyniki projektu.....	7

1. Cel projektu

Celem projektu było opracowanie aplikacji webowej umożliwiającej gromadzenie, zarządzanie i udostępnianie fotografii archiwalnych. System miał wspierać zarówno publiczne przeglądanie zasobów, jak i pracę użytkowników odpowiedzialnych za dodawanie oraz moderowanie materiałów. Istotnym założeniem było rozdzielenie warstwy metadanych od przechowywania plików zdjęć oraz przygotowanie aplikacji do dalszej rozbudowy o kolejne funkcje archiwalne.

2. Zakres funkcjonalności

Zakres projektu obejmuje funkcje związane z obsługą użytkowników, zarządzaniem materiałami fotograficznymi oraz publicznym udostępnianiem archiwum. System pozwala na rejestrację i logowanie użytkowników, dodawanie zdjęć wraz z metadanymi, organizowanie materiałów w kategoriach, moderację treści przez administratora oraz przeglądanie zbiorów przez użytkowników publicznych. Aplikacja udostępnia również wyszukiwanie, filtrowanie oraz prezentację zdjęć na mapie.

2.1. Charakterystyka użytkowników aplikacji

W systemie wyróżniono trzy podstawowe role użytkowników:

- **Przeglądający** - użytkownik niezalogowany lub zalogowany, którego głównym zadaniem jest przeglądanie publicznie dostępnych materiałów. Może on otwierać listę zdjęć, przejść do widoku szczegółowego, korzystać z wyszukiwania, filtrowania oraz widoku mapy.
- **Twórca** - użytkownik posiadający konto, który może dodawać nowe zdjęcia do archiwum oraz zarządzać własnymi materiałami. Do jego uprawnień należy edycja metadanych własnych zdjęć, usuwanie własnych rekordów oraz uzupełnianie informacji takich jak lokalizacja, data i kategoria.
- **Administrator** - odpowiada za nadzór nad systemem i moderację danych. Może zarządzać użytkownikami, blokować twórców, przeglądać nowe zdjęcia dodane od ostatniego logowania oraz edytować lub usuwać zdjęcia w ramach moderacji.

2.2. Szczegółowy opis funkcji

Funkcje systemu można podzielić na trzy główne grupy.

Funkcje publiczne

- przeglądanie listy wszystkich dostępnych zdjęć,
- otwieranie widoku szczegółowego zdjęcia,
- wyszukiwanie zdjęć po frazie,
- filtrowanie po kategorii, lokalizacji i dacie,
- prezentacja zdjęć na mapie na podstawie współrzędnych,
- przeglądanie archiwum zarówno przez użytkowników zalogowanych, jak i niezalogowanych.

Funkcje twórcy

- rejestracja i logowanie do systemu,
- dodawanie nowych zdjęć wraz z metadanymi,
- wskazywanie lokalizacji tekstowej oraz współrzędnych,
- przypisywanie zdjęcia do kategorii,
- edycja własnych zdjęć,
- usuwanie własnych zdjęć,
- przeglądanie listy własnych materiałów.

Funkcje administratora

- przegląd listy zarejestrowanych użytkowników,
- blokowanie wybranych twórców,
- przegląd zdjęć dodanych od poprzedniego logowania administratora,
- edycja metadanych zdjęć w ramach moderacji,
- usuwanie zdjęć ze wspólnego archiwum,
- wykonywanie działań administracyjnych z poziomu panelu administratora oraz wspólnego widoku zdjęcia.

2.3. Wymagania niefunkcjonalne

- System umożliwi korzystanie z responsywnego i intuicyjnego interfejsu użytkownika spełniającego kryteria standardu WCAG 2.1 na poziomie AA.
- System umożliwia dostosowanie wyglądu do wybranego przez użytkownika stylu jasny/ciemny/kontrastowy (dla osób słabo widzących).

3. Realizacja projektu

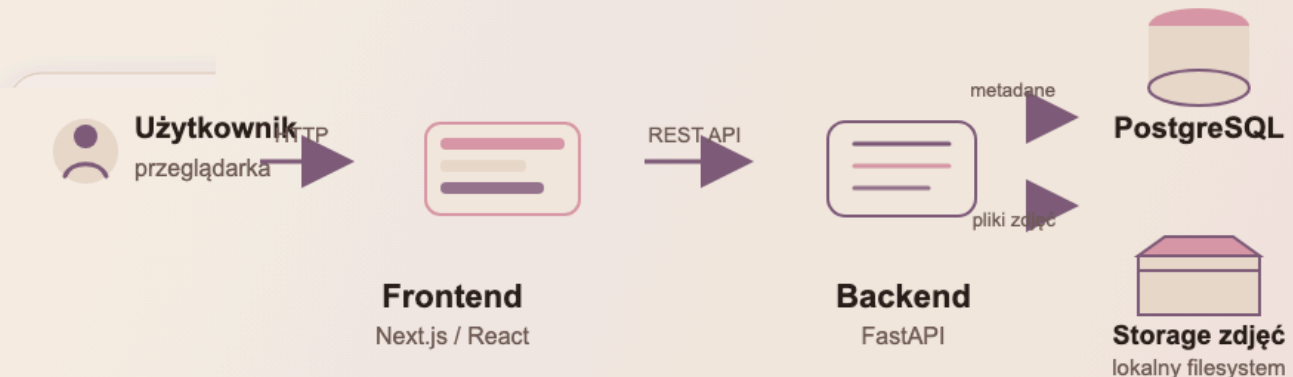
3.1. Architektura systemu

Aplikacja została zrealizowana w architekturze rozdzielającej warstwę prezentacji, warstwę logiki aplikacyjnej, warstwę danych oraz warstwę przechowywania plików. Frontend odpowiada za interfejs użytkownika i komunikację z backendem przez REST API. Backend realizuje logikę biznesową, autoryzację, operacje na zdjęciach oraz dostęp do danych. PostgreSQL przechowuje metadane i relacje pomiędzy obiektami, natomiast same pliki zdjęć są przechowywane poza bazą danych, na lokalnym filesystemie backendu.

Taki podział upraszcza rozwój systemu i pozwala niezależnie rozwijać interfejs użytkownika, logikę aplikacyjną oraz model danych. Rozdzielenie metadanych od plików zdjęć ogranicza obciążenie bazy danych i ułatwia późniejszą rozbudowę rozwiązania o inny mechanizm storage, np. zgodny z S3.

Architektura systemu

Frontend komunikuje się z backendem przez REST API; backend zarządza metadanymi i plikami zdjęć.



Rysunek 1: Diagram architektury systemu

3.2. Wykorzystane technologie

W projekcie zastosowano osobne technologie dla warstwy backendowej, frontendowej i danych. Taki podział pozwolił dobrać narzędzia zgodnie z rolą danej części systemu: backend odpowiada za logikę, frontend za interfejs użytkownika, a baza danych za trwałe przechowywanie metadanych i relacji.

3.2.1. Back-End

Warstwa backendowa została zrealizowana w języku Python z wykorzystaniem frameworka FastAPI. Odpowiada ona za udostępnianie REST API, obsługę uwierzytelniania, autoryzację użytkowników oraz realizację operacji na zdjęciach, kategoriach i użytkownikach.

W backendzie wykorzystano również:

- **SQLAlchemy** do mapowania obiektowo-relacyjnego i pracy z bazą danych,
- **Alembic** do obsługi migracji schematu bazy danych,
- **Uvicorn** jako serwer aplikacyjny ASGI,
- **PyJWT** i mechanizmy hasel do obsługi logowania i tokenów dostępowych,
- **uv** do zarządzania zależnościami i uruchamiania środowiska Pythona.

3.2.2. Front-End

Warstwa frontendowa została zrealizowana w technologii Next.js z użyciem React i TypeScript. Odpowiada za prezentację interfejsu użytkownika, obsługę formularzy, komunikację z backendem oraz renderowanie widoków publicznych i prywatnych.

Frontend obejmuje:

- publiczny widok archiwum,

- widok szczegółowy zdjęcia,
- moduł logowania i rejestracji,
- panel twórcy,
- panel administratora,
- widok mapy,
- mechanizmy wyszukiwania i filtrowania.

Zastosowanie TypeScript ułatwia kontrolę typów danych przesyłanych pomiędzy frontendem i backendem oraz zmniejsza ryzyko błędów integracyjnych.

3.2.3. Baza Danych

Jako bazę danych wykorzystano PostgreSQL. Rozwiązanie relacyjne dobrze odpowiada strukturze danych projektowych, w której istotne są powiązania pomiędzy obiektami oraz spójność modelu.

3.3. Baza danych

W systemie zastosowano relacyjną bazę danych PostgreSQL. Model danych został zaprojektowany tak, aby oddzielić przechowywanie metadanych od przechowywania samych plików zdjęć. Baza zawiera informacje o użytkownikach, zdjęciach oraz kategoriach, natomiast pliki obrazów są zapisywane na lokalnym filesystemie backendu.

Najważniejsze encje systemu to:

- users – konta użytkowników, role, status blokady oraz dane potrzebne do logowania,
- photos – metadane zdjęć, informacje o autorze, kategorii, lokalizacji, dacie i ścieżce do pliku,
- photo_categories – słownik kategorii wykorzystywany do organizacji materiałów.

Relacje pomiędzy encjami są proste:

- jeden użytkownik może być właścicielem wielu zdjęć,
- jedno zdjęcie należy do jednej kategorii,
- kategoria może posiadać kategorię nadrzędną, co pozwala rozszerzyć strukturę do prostego modelu hierarchicznego.

Model bazy danych

Główne encje systemu: użytkownicy, zdjęcia oraz kategorie zdjęć.



Rysunek 2: Schemat bazy danych

3.4. Server aplikacyjny

Server aplikacyjny został zrealizowany jako backend FastAPI. Odpowiada on za udostępnianie endpointów REST, obsługę autoryzacji i uwierzytelniania użytkowników, walidację danych wejściowych oraz realizację logiki związanej z zarządzaniem zdjęciami i użytkownikami.

Warstwa backendowa została podzielona na moduły odpowiadające za:

- endpointy API,
- schematy request/response,
- modele danych,
- repozytoria dostępu do bazy,
- logikę pomocniczą, np. obsługę storage zdjęć.

Backend udostępnia również dokumentację API w Swagger UI, co upraszcza testowanie i prezentację endpointów podczas demonstracji projektu.

3.5. Aplikacja webowa

Aplikacja webowa została zrealizowana jako frontend Next.js z wykorzystaniem React i TypeScript. Interfejs użytkownika obsługuje zarówno funkcje publiczne, jak i funkcje wymagające zalogowania.

W aplikacji zaimplementowano następujące widoki:

- publiczną listę zdjęć,
- widok szczegółowy zdjęcia,
- ekran logowania i rejestracji,
- panel twórcy z listą własnych materiałów,
- formularze dodawania i edycji zdjęć,
- panel administratora,
- widok mapy,
- mechanizmy wyszukiwania i filtrowania.

Frontend komunikuje się z backendem przez wywołania HTTP do REST API. W warstwie interfejsu zastosowano wspólne komponenty odpowiedzialne za prezentację zdjęć, formularze, panele administracyjne oraz nawigację pomiędzy głównymi modułami aplikacji.

3.6. Deploy

Projekt został przygotowany do uruchamiania lokalnego z wykorzystaniem Docker Compose. W konfiguracji uruchamiane są trzy podstawowe usługi:

- frontend,
- backend,
- PostgreSQL.

Backend podczas startu:

- instaluje zależności,
- wykonuje migrację bazy danych,
- uruchamia serwer aplikacyjny.

Frontend podczas startu:

- instaluje zależności,
- uruchamia aplikację Next.js w trybie developerskim.

Takie podejście upraszcza uruchomienie projektu na nowym środowisku i pozwala odtworzyć cały stos aplikacyjny przy użyciu jednej komendy.

4. Wyniki projektu

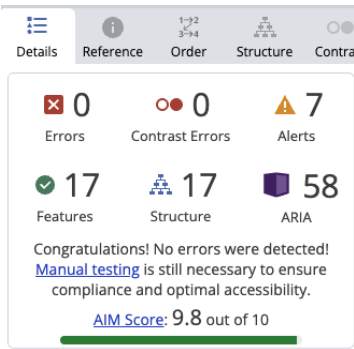
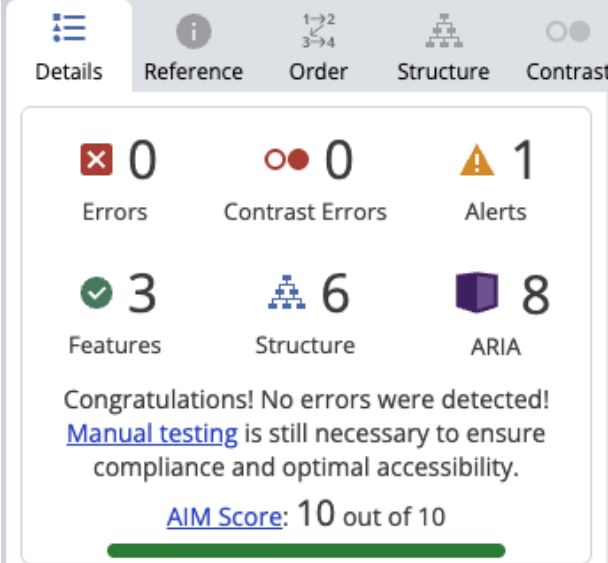
W wyniku realizacji projektu powstała działająca aplikacja webowa do zarządzania i udostępniania fotografii archiwalnych. System obejmuje zarówno część publiczną, jak i funkcje przeznaczone dla twórców oraz administratora.

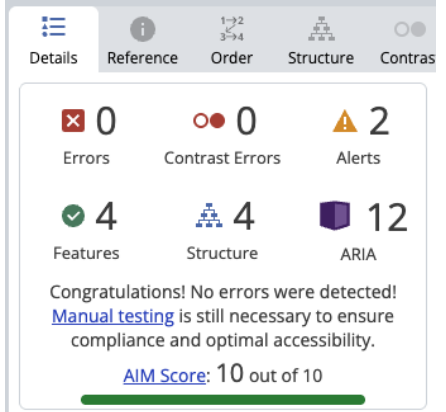
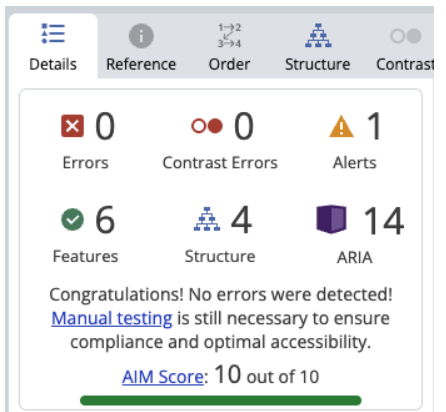
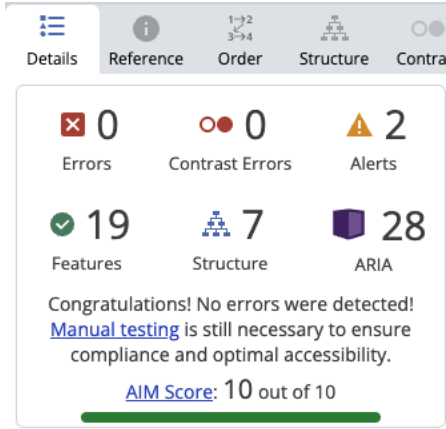
Zrealizowano w szczególności:

- obsługę rejestracji i logowania użytkowników,
- podział na role użytkowników,
- dodawanie, edycję i usuwanie zdjęć,

- organizację zdjęć w kategoriach,
- publiczne przeglądanie archiwum,
- widok szczegółowy zdjęcia,
- wyszukiwanie i filtrowanie,
- prezentację zdjęć na mapie,
- funkcje moderacyjne administratora,
- dokumentację API oraz możliwość prezentacji danych z bazy.

Interfejs aplikacji został przygotowany jako responsywny i dostępny cyfrowo. Przyjęto zgodność ze standardem WCAG 2.1 na poziomie AA, obejmującą m.in. poprawną semantykę, obsługę klawiatury, odpowiedni kontrast, widoczność fokusu oraz czytelne komunikaty formularzy. System wspiera również zmianę wariantu wizualnego interfejsu pomiędzy trybem jasnym, ciemnym i wysokokontrastowym.

Nazwa widoku	Ścieżka aplikacji	screen WAVE
Strona główna archiwum	/	 Details Reference Order Structure Contra 0 Errors 0 Contrast Errors 7 Alerts 17 Features 17 Structure 58 ARIA Congratulations! No errors were detected! Manual testing is still necessary to ensure compliance and optimal accessibility. AIM Score: 9.8 out of 10
Publiczny widok szczegółowy zdjęcia	/all_photos/[photoId]	 Details Reference Order Structure Contrast 0 Errors 0 Contrast Errors 1 Alerts 3 Features 6 Structure 8 ARIA Congratulations! No errors were detected! Manual testing is still necessary to ensure compliance and optimal accessibility. AIM Score: 10 out of 10

Logowanie	/login	
Rejestracja	/register	
Panel twórcy / lista własnych zdjęć	/photos	

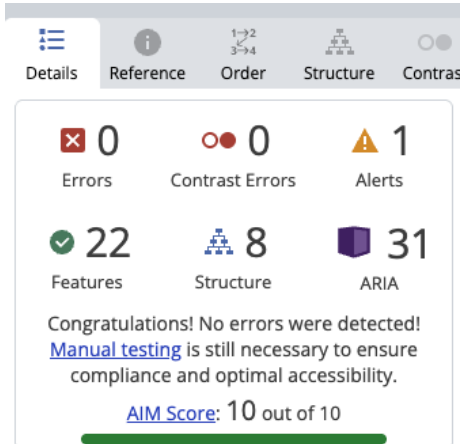
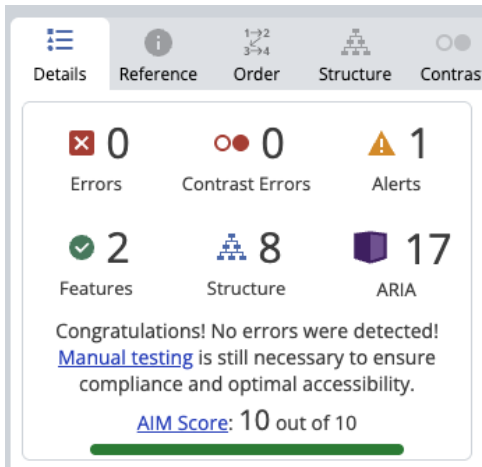
Szczegóły i edycja zdjęcia twórcy	/photos/[photoId]	
Panel administratora	/admin	

Tabela 1: Wyniki analizy WAVE dla każdej strony